实验原理：RSA 共享素因子攻击

—— 素数是怎么来的，为什么「撞车」是灾难，以及怎么在一千个数里一次算完

0. 一页速览

#	缺陷类型	痕迹	用什么检测	代价	真实世界里
1	**共享素因子**	gcd(n_i, n_j) 落在 (1, n) 之间	两两 gcd	**O(n^2)**	存在（约 0.5%）
2	模数复用	gcd 给出 n 本身	排序去重	O(n log n)	**罕见**
3	完全平方 $n = p^2$	isqrt(n)² == n	isqrt	**O(n)，几毫秒**	**罕见**
4	小素因子	和「小素数之积」有公因子	一次大数 gcd	O(n)	本实验语料中为 0
5	相近素数	$a^2 - n$ 是完全平方	Fermat 分解	每台几百步	存在（高位熵不足）
6	结构化素数（ROCA）	素数有隐蔽结构	专用工具（格约简）	——	存在（2017）

这张表是本实验的骨架。 要注意的是最右两列：

- 不同的 bug 留下**不同的痕迹**，没有一种工具能全看见；

1. 起点：RSA 的三个数字

1.1 从欧拉定理出发

第一步：欧拉函数 φ(n) 是「小于 n 且与 n 互素的正整数个数」。

对我们关心的 n = p·q（p、q 是两个不同素数），可以数出来：

```
在 1…pq 里，与 p 不互素的有 q 个（p，2p，…，qp）
            与 q 不互素的有 p 个（q，2q，…，pq）
            两者只有一个交点（pq 本身）

ϕ(pq) = pq − q − p + 1 = (p − 1)(q − 1)
```

第二步：欧拉定理。若 gcd(a, n) = 1，则

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

第三步：把这条件改造成「能加密解密」。

取 e 满足 gcd(e, φ(n)) = 1。由 Bézout 恒等式，存在 d 使

$$e \cdot d \equiv 1 \pmod{\phi(n)} \qquad \text{也就是} \quad e \cdot d = 1 + k \cdot \phi(n)$$

于是对任意与 n 互素的 m：

$$(m^e)^d = m^{e \cdot d} = m^{1 + k \cdot \phi(n)} = m \cdot (m^{\phi(n)})^k \equiv m \cdot 1^k \equiv m$$

这就是 RSA 能工作的全部理由。

1.2 三个数字，两种性质

	符号	怎么来的	公开吗
模数	n = p·q	两个随机素数相乘	**公开**
公开指数	e	随便挑一个与 φ(n) 互素的数（常用 65537）	**公开**
私钥指数	$d = e^{-1} \bmod \varphi(n)$	**算出来的**	保密

关键结构：d 是算出来的，不是选出来的。

这句话是整个实验的地基。它意味着：

- 只要攻击者知道 **φ(n)**，就能算出 d；
- 只要攻击者知道 **p 和 q**，就能算出 φ(n)；
- 只要攻击者能**分解 n**，就有了 p 和 q。

所以 RSA 的安全性**等价于**分解 n 的困难性 —— 这是教科书上的标准结论。

但本实验攻击的根本不是这条路。

1.3 本实验走的是另一条路

我们**一次也不分解**任何 1024 位的数。我们利用的是：

> 攻击者的目标不是「分解 n」，而是「拿到 φ(n)」。 而 φ(n) 的来源不止一条。

如果两台设备的模数共享一个素因子：

```
n₁ = p · q₁
n₂ = p · q₂
gcd(n₁, n₂) = p          ← 一次 gcd，代价是微秒级
```

拿到 p 之后，$q_1 = n_1 / p$，$\phi(n_1) = (p-1)(q_1-1)$，$d_1 = e^{-1} \bmod \phi(n_1)$。

私钥到手。全程没有任何「分解」发生。

一次 512 位素数的 gcd，在 Python 里大约 **12 微秒**； 分解一个 1024 位的合数，在本机上的代价远超宇宙年龄。 **差的是天文数字的量级，而这个差距完全来自「两台设备用了同一个素数」 这一个事实。**

---

2. 素数从哪来：随机性是怎么变成密钥的

2.1 素数不是「想出来」的，是「抽」出来的

生成一个 512 位素数，标准做法是：

```
循环：
    抽一个 512 位的随机数 r，把最高位置 1（保证位长）、最低位置 1（保证奇
    用素性检测（如 Miller-Rabin）判断 r 是不是素数
    是 ⟶ 返回 r
    否 ⟶ 再抽一个
```

根据素数定理，512 位附近大约每 $\ln(2^{512}) \approx 355$ 个数里有一个素数。 所以平均抽 355 次能中一次 —— **这个循环的每一次迭代，都依赖随机数的质量。**

2.2 熵池没攒够的时候

现代系统的随机数来自「熵池」—— 由硬件中断、时钟抖动、磁盘 I/O 时序等 不可预测的事件填充。**问题出在开机后的头几秒。**

Linux 内核在熵池达到某个估计值之前，`/dev/urandom` 也**不会阻塞** —— 它照样给你字节，只是这些字节的可预测性比你想象的高。 嵌入式设备启动极快、中断很少（没有键盘、没有鼠标、没有磁盘）， 熵池积累特别慢。

如果两台设备在这个窗口里生成了密钥，它们抽到的「随机数」 很可能来自同一个很小的种子空间。**两个素数就撞上了。**

2.3 三种撞车方式，三种不同的痕迹

「随机数出问题」不是一个单一的故障，它会以几种不同的方式显形：

出了什么事	留下的痕迹	检测手段
两台设备抽到了**同一个素数**	两个模数有非平凡公因子	两两 gcd
一台设备**把同一个素数用了两次**	n 是完全平方	isqrt
两台设备抽到的素数落在**同一个窄区间**里	p、q 挨得很近	Fermat 分解
设备出厂时**克隆了同一份密钥**	模数完全相同	排序去重

注意最后一行：这是唯一一种「异常但攻击者拿不到私钥」的情况。 详见 §3.2。

2.4 概率到底有多低

有人会想：512 位的素数有大约 $2^{512} / \ln(2^{512}) \approx 10^{152}$ 个， 两台设备撞上同一个的概率应该低到可以忽略吧?

这个直觉忽略了「随机数不是真的随机」。 概率计算假设两次抽数是独立的、 均匀的。而真正的风险来自**独立性被破坏**：

- 熵池不足时，两次「随机」抽数的实际熵可能只有几十比特，而不是 512 比特；
- 设备克隆时，两次抽数是**完全相同的**，概率不是什么「10^{-152} 分之一」， 而是**1**。

2012 年那篇论文的实测结果说明了这一点（见 §7.2）： 在 580 万张 TLS 证书里，**0.50%** 可以被 gcd 直接恢复私钥。 如果随机性真的是理想的，这个数字应该是**零**。

> 📌 所以本实验的攻击，攻击的不是「运气不好」， 而是**随机数发生器在特定条件下的确定性行为。**

---

3. 弱密钥的谱系

§0 那张表的六行，展开讲清楚。

3.1 共享素因子 —— 本实验主线

$$\gcd(n_1, n_2) = p$$

- **痕迹**：两个模数之间有一个非平凡的公因子
- **检测**：两两 gcd，看结果落不落在 $1 < g < n$
- **危害**：两台设备的私钥**同时**泄露
- **关键词**：**跨设备** —— 单看任何一台设备，它都是完全正常的

最后一条是这一类最反直觉的地方。你没有权限去检查设备的随机数发生器， 你也无法从**一个**模数里看出任何异常 —— 它就是一个标准的、 两个 512 位素数之积。

缺陷不在任何一台设备里，缺陷在它们之间的关系里。

3.2 模数复用 —— 异常，但拆不开

$n_1 == n_2$　　（两个模数完全相同）

- **痕迹**：两个模数一模一样，gcd 给出 n 本身
- **检测**：排序去重，或注意到 g == n
- **危害**：**不能分解**，但同样是严重缺陷

为什么不能分解？ 因为 gcd(n，n) = n，而 n 本来就是公开的。 这个运算没有给你任何**新信息**。

那它为什么还是严重缺陷？

> 两台设备共享同一把私钥，意味着**任意一方可以伪装成另一方**。 这直接破坏了「不可否认性」—— 设备 A 发出的签名，设备 B 也能发， 事后无法区分是谁做的。而且如果其中一台被物理攻破，另一台**同时**失守。

「异常」和「可分解」是两件事。 这是本实验最重要的一个区分， 也是任务三最常见的错误来源，更是你在报告里**必须分两栏写**的理由 —— 把两者合成一个数字，真正危险的那几台就被稀释掉了。

3.3 完全平方 n = p²

$n = p \times p$

- **痕迹**：n 是一个完全平方数
- **检测**：isqrt(n) 之后平方回去看等不等于 n
- **危害**：私钥泄露

这一类对两两 gcd 完全沉默。 为什么？

$\gcd(p^2,$ 别的模数$) = 1$　　（别的模数都是独立随机的，没有一台持有 p）

它和语料里**其他每一个模数**的 gcd 都是 1，而「自己和自己」不算一对。 所以扫描跑完，它**一条命中都没有**。

> 📌 这是本实验最省时省力的一个知识点。 **发现它靠的是「知道还有这一类」，不是靠算力。**

> 而它极其便宜。本机实测（1,000 个模数，连测三轮）：
>
> | 做法 | 耗时 |
> |---|---|
> | 两两扫描（499,500 次 gcd） | 6.72 ~ 7.10 秒 |
> | 开方检查（1,000 次 isqrt） | **2.2 ~ 4.0 毫秒** |
>
> **快约两千倍（实测 1704 ~ 3097 倍）。** 不是难找，是容易忘。（绝对秒数随机器和负载变，三个数量级的差距不变。）

φ(p²) 有陷阱，见 §5.2。

3.4 小素因子（本实验不做，只讲原理）

n 有一个 32 位以内的小因子。检测方法是把前面若干个小素数乘成一个大数 P，然后 `gcd(n, P)`。这又是**另一套**技术，和两两扫描、开方检查都不重叠。

这一类在真实语料里的命中数是 0。 有一份 100,000 个模数的真实清单，实测：

检测项	命中
重复模数	**0 组**
完全平方	**0 个**
含 < 100,000 的素因子	**0 个**

所以本实验**没有**把这一族做进语料 —— 造不出现实的数据：要让它既带一个小因子又保持 1024 位模数，另一个因子得有约 1011 位，没有哪个密钥生成器会这么干；而现实的版本（两个因子都小）会让模数只有 525 位，**光看位长就能挑出来**，检测点就废了。

但这件事本身是重要的一课：

> 一个检测项在真实数据里命中为 0，**不代表它不该做** —— 它代表它**这一次**没有命中。完全平方在真实数据里也是 0，而它的检测代价只有 3 毫秒。
>
> **便宜的正交检测，永远值得加。**

3.5 相近素数 —— Fermat 分解（本实验不做，只讲原理）

> 📖 **这一节请当阅读材料看**：本包学时有限，不动手做这一类。

p 和 q 挨得很近。这时 n 约等于 $((p+q)/2)^2$，可以用**费马分解**。

设 n = p·q，把 p、q 写成 a 加减同一个数：

```
a = (p + q) / 2          b = (q − p) / 2
```

于是

$$n = p \cdot q = (a - b)(a + b) = a^2 - b^2$$

所以只要找到一对 (a, b) 使 $a^2 - n = b^2$，就分解成功了。 a 从 $\lceil \sqrt{n} \rceil$ 开始往上试，每一步检查 $a^2 - n$ 是不是完全平方。

p、q 越接近，a 越贴着 √n。 由均值不等式 $a = (p+q)/2 \geq \sqrt{(pq)} = \sqrt{n}$， 两者之差随 $|p - q|$ 迅速缩小。

真相	值
每步代价（1023 位模数）	约 **2.0 微秒**（一次大整数乘 + 一次 isqrt）
攻破一台「因子间距 G 比特」的设备	步数 ≈ **2^(2G − 514)**

最后一行是本节最重要的一条。它反过来说：

> **步数预算每翻一倍，能多覆盖的间距只有半个比特。**

所以把预算从 4096 步提到 100 万步（**256 倍**的时间）， 只多覆盖 **4 个比特**的间距。拿 256 倍的开销换 4 个比特 —— 这笔账在任何场景下都不划算。

这就是为什么 Fermat 只能用来「顺手查一下」，不能当通用分解法用。 而且这也是为什么**任何检测都必须报出自己的边界**： 一个不报预算的 Fermat 检测，「没找到」这句话是可以被读成 「这类缺陷不存在」的 —— 而它真正的意思是 「在 4096 步以内没找到」。

完全平方和相近素数的起手式一模一样（都是在 `isqrt(n)` 附近找完全平方）， 区别只是完全平方一次就中（$n = p^2$ 时 $b^2 = 0$）。 **两族缺陷「痕迹不同」不等于「检测手段无关」** —— 这一条在这里体现得很干净。

3.6 结构化素数（ROCA）（本实验不做，只简单介绍）

2017 年披露的 CVE-2017-15361。Infineon 的 RSA 密钥生成库 没有直接用随机数选素数，而是用一种**筛法**构造素数 —— 留下的结构让攻击者可以用一种**格约简**的方法恢复私钥。

检测它需要**专用的**工具，和上面所有方法都无关。

---

4. 两两 vs 批量：为什么本实验只做朴素版

4.1 朴素两两比对

直接的想法：所有模数两两配对。

```
对数 = C(n, 2) = n(n−1)/2 ≈ n²/2
```

本机实测（Python 3.14.5 / Windows 10，下面是其中一轮）：

规模 n	对数 C(n,2)	实测耗时	每对耗时	相对上一档
125	7,750	0.097 ~ 0.105 s	12.5 ~ 13.6 µs	—
250	31,125	0.398 ~ 0.452 s	12.8 ~ 14.5 µs	3.78 ~ 4.68 倍
500	124,750	1.609 ~ 1.643 s	12.9 ~ 13.2 µs	3.56 ~ 4.13 倍
1,000	499,500	6.370 ~ 6.421 s	12.8 ~ 12.9 µs	3.88 ~ 3.99 倍

双对数最小二乘拟合：**指数 2.014。** 规模每翻一倍，耗时涨约 **4 倍**（小规模那两档噪声大，看后两档更准）。

> 📌 **注意第三列到第四列的对比**：耗时涨了 65 倍，但**每对耗时几乎没变**（12.5 µs → 12.8 µs）。这说明总耗时的增长**完全来自对数**，不是「单次 gcd 变慢了」——这是 $O(n^2)$ 的**独立证据**，比拟合出来的那个指数更可靠（指数会被噪声带偏）。

外推（按每对 12.8 µs）：

规模	对数	预计用时
2,000	1,999,000	25.5 秒
10,000	49,995,000	10.6 分钟
100,000	4,999,950,000	**约 18 小时**
1,000,000	499,999,500,000	约 74 天

约 18 小时。而 100,000 正是 2012 年那篇论文的规模（见 §7.2）。这在工程上不能接受。

> 📌 **上面每个数都给了一个范围，因为连测三轮它们确实不一样。**唯一稳定的是：指数贴近 2、每对耗时几乎不变、每档涨约四倍。你要记的是这三件事，不是那几个绝对秒数。

> 📌 看「每对耗时」那一列：12.0 → 12.2 µs，只差 1.0 倍。这**不是噪音，是证据**——它说明总耗时的增长**完全来自「对数」**，而不是「单次 gcd 变慢了」。
>
> 这是拟合之外的**第二条独立证据。**两条独立证据指向同一个结论，比单独一个拟合指数可靠得多——拟合指数可能被噪声带偏，「每对耗时恒定」不会。

4.2 正规解法：批量 GCD

真实研究用的是**批量 GCD**（**本实验不要求实现**），大致思路：

把 n 个数两两的 gcd 一次性求出来，用的是**乘积树**的思想：

```
把 n 个数分成两组，各自求出组内所有数的乘积
递归下去，得到一棵二叉树
每次拿「左子树所有数的积」和「右子树所有数的积」做一次 gcd
```

为什么「先乘起来」不会丢掉信息？ 因为 gcd 对乘法是「分配」的：

```
gcd(a·b, c·d) 里包含 a 与 c 的公因子、a 与 d 的公因子……
```

一个共享的素数 p 会出现在积里，所以只要 p 同时存在于左右两棵子树的 某两个数里，这次 gcd 就会把它**整批**带出来。

复杂度可以降到接近 **O(n log²n)** —— 对外推那张表里的 100,000 个模数， 这把十几个小时压到了分钟级。有兴趣的同学可以拿 §7 的加分项去做。

本实验为什么不做批量 GCD，想动手的同学可以自行改代码尝试 —— 本实验不要求。

---

5. 从素因子到私钥

5.1 标准流程

拿到 p（无论是 gcd 给的还是 isqrt 给的）之后：

```
q    = n // p
phi  = (p - 1) * (q - 1)      # ← 一般情形
d    = modinv(e, phi)          # e·d ≡ 1 (mod phi)
```

然后必须做一次往返验证：

```
c  = pow(m, e, n)              # 公钥加密
m2 = pow(c, d, n)              # 私钥解密
assert m2 == m
```

为什么往返验证不能省？ 因为前面的每一步都可能错， 而错的方式往往**不报错**（见 §5.4）。往返验证是「这把私钥是对的」的 **唯一独立证据** —— 它不检查你的推导过程，只检查结果能不能用。

5.2 坑一：φ(p²) 不等于 (p−1)²

在完全平方那条路上，`p == q`。这时通用的 `(p-1)(q-1)` 公式给出的是 `(p-1)²`，而**正确答案**是：

```
φ(p²) = p² − p = p(p − 1)
```

两者差一个 `p − 1`，也就是一个 **512 位的数**。

为什么会算错？ 因为写 `(p-1)*(q-1)` 太自然了 —— 它是标准公式， 在 999 台设备上都是对的。

算错了会怎样？ d 会是一个看起来完全正常的数，**程序不报任何错**， 只是最后解不开。而且它比你想的更难发现：

> 错的 φ 是 `(p-1)²`，真的 φ 是 `p(p-1)`。因为 `(p-1)² < p(p-1)`，错算出来的那个 d **仍然落在正确的区间 `1 < d < φ_true` 里** —— 所以连「d 的范围检查」都不会响。 **这条链上没有任何一步会报错**（详见 §5.4）。

你会在任务四花很长时间怀疑 PKCS#1、怀疑 AES、怀疑会话密钥 —— 而真正的问题在几百行之前。

防线：`task1_math.py --selftest` 的**块【五】**。 它拿一个手算的独立预言机（比如 `φ(7,7) == 42`）直接对答案， **不问过程只问结果** —— 这是全包唯一能在 5 秒内发现它的检查。 （totient 现在是你自己写的，就在 `task1_math.py` 里， 正文的两种情形各是一处填空。）

5.3 坑二：d 可能是负数

`egcd(a, m)` 给出的 Bézout 系数**可能为负**。比如：

```
egcd(3, 7) → (1, −2, 1)          因为 3×(−2) + 7×1 = 1
```

`x = −2` 是完全合法的 Bézout 系数，但它不是「逆元」的规范形式。 归约一下：`−2 mod 7 = 5`，验算 `3×5 = 15 ≡ 1 (mod 7)` ✓。

为什么这个坑特别毒？ 因为它能穿过三道本来应该拦住它的检查：

检查	对 d = −2 的表现
Bézout 恒等式	**照样成立** —— 它本来就是解
`e * d % phi == 1`	**照样为真** —— Python 的 `%` 总是返回非负数
位长看起来正常	**是的** —— 负数的位长也是 1024

唯一能抓住它的检查是范围检查：`0 <= d < m`。

这就是任务一自检里【三】为什么必须单独列一条 —— 它不是锦上添花， 它是这一族错误唯一的一道门。

5.4 小结：三个静默失败

坑	症状	唯一防线
元组赋值拆成两句	不报错，答案全错	任务一 Bézout 自检【一】
`int(n ** 0.5)` 代替 `isqrt`	不报错，一台都找不到	**先自己算应该有几台**
$\phi(p^2)$ 写成 $(p-1)^2$	不报错，**范围检查也不响**，最后解不开	自检块【五】的独立预言机
`d` 是负数	不报错，前三道检查都过	范围检查 `0 <= d < m`（自检【三】）

> **共同点：四个都不报错。** 这就是本实验最想让你记住的一件事 —— **静默失败比崩溃危险得多。**
>
> 崩溃会告诉你「哪里错了」。静默失败会在几百行之后 以一个完全无关的症状出现，然后你花两小时改一个本来没错的地方。

---

6. PBP：黑盒里有什么

6.1 三层结构

`homework.pdf.enc.asc` 是一个 PBP 文件（**P**retty **B**ad **P**rivacy —— 本包自带的一种教学用密钥容器，二进制 blob 外面套一层 base64/ASCII 封装， 带 BEGIN/END 头尾标记）。它里面是这样的：

```
RSA 密文（1023 位）          ← RSA 作用在这一块上
IV（16 字节，明文携带）
AES 密文体（16,912 字节）
```

解开之后，RSA 密文那部分是一个 **PKCS#1 v1.5** 加密块：

```
EM = 0x00 || 0x02 || PS || 0x00 || session_key
└──────────────── RSA 作用在这一整块上 ────────────────┘
```

- PS 是**随机填充**，至少 8 字节，不含 0x00；
- `session_key` 是真正的密钥（这里是一个 AES-256 的密钥）。

为什么要包这么多层？ 因为 RSA **直接加密大文件是不行的**：

RSA	AES
慢	快
一次只能加密**不到 1024 位**	一次加密 128 位一块，任意长
模幂运算	位运算

这叫**混合加密**：用 RSA 保护**会话密钥**，用会话密钥保护**数据**。 这是 TLS、PGP、SSH 全都采用的结构。

6.2 解密就是倒过来走

```
1. RSA 私钥解密：EM = c^d mod n
2. 剥掉 PKCS#1 填充：从 EM 里取出 session_key
3. AES-CBC 解密：明文 = AES⁻¹(session_key, IV, 密文体)
4. 去掉 PKCS#7 填充
```

6.3 ★ 反向确认要验整块 EM ★

第 1 步做完之后，应该有一个**反向确认**：

```
pow(os2ip(em), e, n) == c            # ✓ 正确
pow(os2ip(session_key), e, n) == c   # ✗ 永远是 False
```

为什么要验? 因为有几把私钥是**错的**（模数重复的那两台算不出私钥， 其他不相关的设备更算不出）。要判断「这把钥匙是不是收件方的」， 只检查填充格式是不够的 —— 极小概率下一把错的钥匙会碰巧产生合法填充。

为什么这个坑特别浪费时间? 因为用 session_key 去加密， 对一把**完全正确**的私钥也会返回 False。你会以为 d 错了， 然后回去检查 φ、检查 modinv、检查 egcd —— 全都没错。

> 📌 记住这条规律：**RSA 作用在整个 EM 块上，不是作用在里面的数据上。** 想「反向验证一下」的时候，要验的是**它实际处理过的那个东西**。

6.4 为什么 SHA-256 是唯一标准

解出来的 PDF 有 **16,896 字节**。这个数字不是随便定的 —— 它是明文的确切长度。

「PDF 能打开」不是验收标准。 举一个真实的例子：如果 AES 解密时 用了保留 PKCS#7 填充的版本，会多出 1~16 字节的填充：

```
明文 = ...PDF 内容... || 0x0A 0x0A 0x0A 0x0A 0x0A 0x0A 0x0A 0x0A 0x0
```

PDF 阅读器照样打得开（它会忽略尾部的垃圾字节）， 但字节数不对，SHA-256 也不对。

这是一个「**看起来成功了**」的失败 —— 最需要警惕的一类。 所以验收标准必须是 **SHA-256 逐字符相同**：

```
d3c1c5b3b6bc7577ff19f6aa1a21e37ad473e9d1b6b5997450e80f2da3dc13d5
```

SHA-256 的性质保证了：改一个比特，哈希就完全不同，而且**不可能**构造出 另一份内容不同、哈希相同的文件。所以只要哈希对上，内容就对上了。

---

7. 真实世界的三个案例

7.1 Debian OpenSSL，2008

发生了什么。 2006 年 9 月，一位 Debian 的 OpenSSL 包维护者 为了消除 Valgrind / Purify 这类内存检查工具报告的两行 「未初始化内存读取」警告，把 `md_rand.c` 里两行向熵池混合数据的代码 **注释掉了。**

他的本意是消除误报。但那两行是**唯一**的熵来源 —— 剩下的只有一个进程 ID（PID），而 Linux 的 PID 上限是 32768。

一个值得记下的细节：这位维护者**知道自己没完全看懂这段代码**， 他把疑问发到了 openssl-dev 邮件列表。列表上的回复是 「可以删掉那行 `MD_update`」—— 回复者也没有仔细看讨论的是哪一个调用。

后果。 Debian 及其衍生发行版上生成的密钥，每种类型只可能有 大约 **32,767 种。**攻击者把所有可能的密钥**全部预计算一遍**， 就能解开这几年的任何 TLS、SSH、OpenVPN 流量。

时间线。 2006 年 9 月引入，2008 年 5 月由 Luciano Bello 发现 （CVE-2008-0166）—— 这个改动在公开的代码仓库里**躺了 20 个月**， 期间无数人看过。

教训。 这是一个**代码审查**问题。一个非密码学专家为了让一个检查工具 「闭嘴」，改掉了密码学代码的核心部分；而他的疑问在邮件列表上 被同样没细看的人「批准」了。 **「没人理解这段代码」和「没人发现问题」，可以同时成立。**

7.2 Mining Your Ps and Qs，2012

本实验正是来自这篇论文（USENIX Security 2012 Best Paper， Heninger、Durumeric、Wustrow、Halderman）。

做了什么。 他们做了一次空前规模的公钥普查：

- 从 1,280 万台主机上收集了 **580 万张**唯一的 TLS 证书
- 从 1,020 万台主机上收集了 **620 万个**唯一的 SSH 主机密钥

然后 —— **跑批量 GCD**。

发现了什么：

结果	TLS	SSH
因共享素因子而**可直接恢复私钥**	**0.50%**	0.03%
以某种可疑方式共享密钥（含默认密钥、弱熵）	5.57%	9.60%

（另有 1.03% 的 SSH 主机因 DSA 签名随机数重用而泄露私钥。）

这些设备绝大多数是**嵌入式网络设备** —— 路由器、防火墙、服务器管理卡， 涉及 54 家厂商。它们的问题是共同的：**开机后极短时间内就生成密钥**， 而 Linux 内核此时熵池还没攒够，种子里实际只有秒级的时间戳。

为什么重要。 这是第一次有人证明，这个问题在真实世界里**大规模存在**， 而不只是理论上的可能。0.50% 听起来很小 —— 但 580 万张证书的 0.50% 是**两万九千台**设备。

> 📎 本实验的语料是这篇论文数据的**合成小规模版本**， 缺陷类型与文件格式都与之一致。真实数据的规模与耗时见《教师参考》。

7.3 ROCA，2017

CVE-2017-15361。Infineon 的 RSA 密钥生成库用一种**筛法** （而不是直接随机试）构造素数，留下的结构让攻击者可以用**格约简** 恢复私钥。影响面估计上亿张智能卡与 TPM。

为什么放在这里：因为它是 §0 那张表里**最右一列**的极端例子 —— gcd 看不见它，开方看不见它，Fermat 分解也看不见它。 检测它需要专门的工具。

「你没发现」不等于「不存在」。 这句话在本实验里会变成一个 具体的要求：你的扫描报告必须写清楚**它覆盖了什么、没覆盖什么** —— 因为两两 gcd 对完全平方是**完全沉默**的，而这不是它的错， 是任何一套检测都有的边界（§3.5 结尾）。

---

8. 防御：能不能在生成时就发现

三条路线，按成本从低到高。

8.1 修随机数

做法：用 `getrandom(2)` 这类会**阻塞**到熵池充足的系统调用， 而不是 `/dev/urandom`；或者内置硬件随机数发生器。

优点：从根上解决。 **缺点**：开机变慢（要等熵），嵌入式厂商有商业动机不做。

8.2 部署时自检

做法：设备生成 n 之后，把 n 送进一个中心数据库比对， 发现重复就重新生成。

技术上是有效的 —— 但它会遇到三个非技术问题：

1. **谁出钱。**数据库要人维护、要能承受全球设备的写入量。
2. **谁愿意交。**把 n 上传到中心库，等于承认「我的密钥可能有问题」。 厂商的法务部门通常第一个反对。
3. **责任怎么分。**如果中心库本身泄露了，谁负责?

> 📌 **技术上可行、组织上不可行**，是密码学工程里最常见的一种失败。

8.3 事后审计

做法：定期收集全网公开的公钥，跑 gcd / 开方检查 / Fermat / ROCA 检测。发现问题就吊销、换证。

优点：不需要任何人配合 —— 公钥本来就是公开的。 **缺点**：**事后。**漏洞被发现的时刻，可能已经过去了几年 （Debian 那个案例是 20 个月）。

本实验的任务三就是这一条。 你写的那个扫描器， 在真实的运维流程里对应的正是「定期扫描自己的设备清单」这个环节 —— 而且真实世界第一次抓到这类漏洞（2012 年那篇论文）用的就是同一套方法。

> 📌 **顺带说清楚一个落差**：真实的审计会**成套上工具** （gcd + 开方 + Fermat + ROCA），本实验只做了前两套。 这不是「本实验的方法不够用」，而是「**任何一套工具都有边界**」—— 这句话你在报告里要能说清楚（§3.5 结尾）。

8.4 最彻底的防御

> 换掉算法。

如果换成**后量子**的密钥封装机制（ML-KEM 等），共享素因子这个失败模式 **根本不存在** —— 因为它压根不依赖「两个大素数之积」这个结构。

---

9. 回到实验：四个任务分别在验证哪一条原理

任务	在验证
一　`egcd` / `modinv` / `totient`	§1.1 第三步：$d = e^{-1} \bmod \phi(n)$ 是**算**出来的；§5.2 φ 的两种情形
二　拆开一对	§1.3：一次 gcd 就能拿到 p，不需要分解

三　扫描全部	§3.1~3.3：三类缺陷、三种痕迹、三套检测；「异常」≠「可分解」
四　解开 PDF	§6：混合加密的三层结构；SHA-256 是唯一验收标准

> §4（$O(n^2)$ 的代价）、§3.5（Fermat 与检测边界）不在动手范围内， 但它们的结论要写进报告 —— 学时有限，本包只做最经典的两套手段。

---

10. 三条可以带走的结论

第一条：RSA 的数学没有坏，坏的是随机数。

从语料里能算出的所有事实，都指向同一个地方 —— 密钥**生成**环节的随机性。RSA 的加密、解密、填充， 每一步都是教科书标准做法，没有一个比特是错的。

> 这意味着：**你没法通过「把 RSA 用得更标准」来解决这个问题。** 它不在算法里，在流程里。

第二条：「检测不到」不等于「安全」。

Fermat 只覆盖 263 比特以内的因子间距；**两两 gcd 对 `n = p²` 完全沉默**； ROCA 那一类谁都看不见。

> 一个诚实的审计报告，必须同时写清楚**检测了什么**和 **没检测什么**。只写「未发现问题」的报告， 在工程上是**有害**的 —— 因为它会被读成「安全」。
>
> 这也是课程思政里那个「技术诚实」的具体落点。 **本实验里它有一个具体的落点**：你跑完两两 gcd 之后， 还得单独扫一遍开方 —— 因为「gcd 没报问题」覆盖的根本不是全部。

第三条：静默失败比崩溃危险。

本实验里有四个坑，**全都不报错**： 元组赋值拆成两句、`int(n ** 0.5)`、$\phi(p^2) = (p-1)^2$、负的 `d`。

> 它们会在几百行之后以一个完全无关的症状出现。 对付它们的唯一办法是：**主动去问「如果这里错了，我怎么知道？」**
>
> 任务一自检里的【三】、任务三里的「先自己算应该有几台」、 任务四里的 SHA-256 核对 —— 三个都是这个问题的答案。
